SYSTEM ANALYSIS DOCUMENTATION (SAD)

Project Name: ORION - Orchestrated Reimbursement Intelligence Operations Network

Version: 1.0

Table of Contents

Introduction 3
- Purpose 3
- Background 3

System Architecture & Design 5
- High-Level Architecture 5
- LLM as Service Layer 6
- Dependency Diagram 6
- Sequence Diagram 8
- Technology Stack 8

Key Data Flows & Database Schema 10
- Data Flow Workflow 10
- Normalised Database Schema (ERD) , 3NF 11

Functional Requirements & MVP Scope 12
- Minimum Viable Product Features 12
- Non-Functional Requirements 13
- Out of Scope / Future Enhancements 13

Monitor, Evaluation, Assumptions & Dependencies 14
- Technical Evaluation 14
- Grayscale Rollout & A/B Testing Strategy 14
- Emergency Rollback (ER) & Golden Release Strategy 14
- Priority Matrix 14
- Monitoring 15
- Assumptions 16
- External Dependencies 16
- Project Management & Team Contributions 17
- Project Timeline 17
- Team Members Role 17
- Recommendations 18

Introduction

This System Analysis Documentation (SAD) presents the technical scope and design decisions behind ORION, the Orchestrated Reimbursement Intelligence Operations Network.

Purpose

ORION is a strategic response to the problem statement identified in the Product Requirements Document: that enterprise expense reimbursement is slow, inconsistent, and fraud-prone when handled manually. The purpose of this document is to provide a comprehensive technical blueprint for development, testing, and review teams.

The key elements of the system covered in this document are as follows:

- **Architecture:** A server-based platform built on FastAPI with a LangGraph-powered graph orchestration engine, integrating a hybrid deterministic and LLM-based decision pipeline.
- **Data Flows:** How a submitted expense claim travels from an employee's receipt upload through intake, parallel analysis, supervisor routing, and final ledger persistence.
- **Model Process:** The end-to-end workflow of the core features: document ingestion, policy validation, fraud intelligence, and audit recording, modelled as a stateful directed graph.
- **Role of Reference:** This document serves as the primary technical reference for developers, QA engineers, and auditors to understand the full system architecture and component interactions of ORION.

Background

Previous Version: Manual Procurement Workflow

Prior to ORION, enterprise expense reimbursement at mid-to-large organizations was entirely manual. Finance personnel reviewed each claim by cross-checking employee-declared amounts against physical or digital receipts, applied policy rules from institutional memory, and escalated ambiguous cases to managers. As headcount and expense volumes grew, review backlogs accumulated, policy enforcement became inconsistent across reviewers, and duplicate submissions went undetected. There was no structured audit trail, leaving organizations exposed during regulatory reviews. A sample manual workflow for a request like "Here is my receipt for the team's Notion workspace for March" would involve at minimum: 2 email handoffs, a manual policy review, a tedious manual search of historical expenses, and manual data entry, often spanning several business days.

Changes in Major Architectural Components

- Replacement of manual review with a graph-based multi-agent orchestration engine (LangGraph).

- Introduction of a hybrid Deterministic + LLM decision pipeline replacing rule-of-thumb human judgment.
- Establishment of a machine-readable, append-only audit ledger (ledger.json) replacing ad-hoc spreadsheet tracking.
- Introduction of parallel processing branches for simultaneous policy and intelligence analysis.

New Capabilities Introduced in ORION

ORION introduces four key capabilities that fundamentally upgrade the reimbursement workflow:

1. Replacement of manual review with a graph-based multi-agent orchestration engine (LangGraph).
2. Introduction of a hybrid Deterministic + LLM decision pipeline replacing rule-of-thumb human judgment.
3. Establishment of a machine-readable, append-only audit ledger (ledger.json) replacing ad-hoc spreadsheet tracking.
4. Introduction of parallel processing branches for simultaneous policy and intelligence analysis.

Target Stakeholders

Stakeholders	Roles	Expectations
Employee	Submits expense claims with supporting PDF and DOCX receipts via the ORION web interface.	Fast, automated reimbursement decisions with clear feedback on claim status and clarification requests.
Finance Reviewer	Reviews only the claims escalated by ORION's supervisor node; accesses AI reasoning trail for each escalated claim.	Concise escalation queue with full decision context, reducing manual review to edge cases only.
System Administrator	Monitors system health via /api/health, manages configuration, and oversees LLM performance via LangSmith.	Real-time observability dashboards, configurable policy rules, and proactive alerting on LLM degradations.
Development Team	Builds, maintains, and iterates on ORION’s multi-agent orchestration pipeline and API layer.	Clear API contracts, modular LangGraph node architecture, well-documented prompt templates, and LangSmith trace access.

System Architecture & Design

High-Level Architecture

Overview

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              CLIENT TIER                                 │
│                                                                          │
│   Browser (Demo UI — single-page HTML/JS, no build step)                 │
│      • Workflow diagram (boxes + arrows; visited/skipped/terminal)       │
│      • Per-agent JSON drill-down                                         │
│      • LangSmith deep-link                                               │
└──────────────────────────────────────────────────────────────────────────┘
                              │ HTTP/JSON
                              ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                       APPLICATION TIER (FastAPI)                         │
│                                                                          │
│   /                  → static HTML                                       │
│   /api/health        → status                                            │
│   /api/submit        → workflow.invoke + trace ──────┐                   │
│   /api/parse-document→ pypdf / python-docx           │                   │
│   /api/ledger        → ledger.all()                  │                   │
│                                                      ▼                   │
│   ┌──────────────────────────────────────────────────────────────┐       │
│   │              ORCHESTRATION (LangGraph workflow)              │       │
│   │                                                              │       │
│   │   intake → intelligence → policy → ┌─supervisor─┐           │       │
│   │                                    │            │           │       │
│   │                                    ▼            ▼           │       │
│   │                              validation     approval         │       │
│   │                              │      │           │           │       │
│   │                              ▼      ▼           │           │       │
│   │                        request_info approval    │           │       │
│   │                              │      │           │           │       │
│   │                              └──────┴──────► recorder ▶ END │       │
│   └──────────────────────────────────────────────────────────────┘       │
│                                                      │                   │
│   ┌──────────────────────────────────────────┐       │                   │
│   │  LLM SERVICE LAYER (app/llm.py)          │ ◀─────┘                   │
│   │  chat / chat_structured                  │                           │
│   │  retry, backoff, json-mode negotiation,  │                           │
│   │  schema injection, output parsing        │                           │
│   └──────────────────┬───────────────────────┘                           │
│                      │                                                   │
│   ┌──────────────────┴──────────────────────┐                            │
│   │  TOOLS (app/tools/)                     │                            │
│   │  PolicyStore | SubscriptionCatalog |    │                            │
│   │  Ledger | document_parser               │                            │
│   └─────────────────────────────────────────┘                            │
└──────────────────────────────────────────────────────────────────────────┘
                 │                                   │
                 ▼                                   ▼
┌────────────────────────────┐        ┌──────────────────────────────┐
│        DATA TIER           │        │       EXTERNAL SERVICES      │
│  data/policies.json (read) │        │  ILMU GLM-5.1 (HTTPS)        │
│  data/org_subscriptions.json │      │  LangSmith (HTTPS, async ingest) │
│  data/ledger.json (append) │        │                              │
└────────────────────────────┘        └──────────────────────────────┘

Cross-cutting:
  • Configuration:  pydantic-settings + .env (app/config.py)
  • Observability:  @traceable on every agent + every LLM call → single trace tree per /api/submit
  • Schemas:        app/schemas.py — single source of truth for state contracts
  • Testing:        offline stub LLM (tests/conftest.py) | live regression nightly
```

*GLM-5.1 compatible with GPT 5.4 mini

Type	Details
System	*Web-based API platform (Python / FastAPI)*
Architecture	*Server-based monolith with a stateful graph orchestration engine (LangGraph)*
AI Integration	*LLM-as-a-Service via OpenAI API (GPT-5.4 mini) with LangChain abstraction*

Persistence	*Append-only JSON ledger (ledger.json)*
Observability	*LangSmith for LLM trace logging; /api/health for system health monitoring*
Frontend	React 19 + TypeScript single-page application built with Vite

ORION is structured as a server-based system with a single backend client interface: a FastAPI REST server. The core of the system is a LangGraph-powered directed acyclic graph (DAG) that represents the expense claim processing workflow as a series of stateful nodes. The WorkflowState object, a central data structure, is passed between nodes, accumulating intelligence signals, policy violations, and confidence scores as the claim traverses the graph. All data is persisted locally in a JSON ledger. External communication occurs exclusively with the OpenAI API for LLM inference and the LangSmith API for observability traces.

LLM as Service Layer

ORION integrates GPT-5.4 mini as a service layer, not as a monolithic AI module. The LLM is invoked by two discrete agent nodes within the LangGraph workflow: the Intelligence Node (responsible for fraud signal analysis and receipt cross-checking) and the Supervisor Node (responsible for reasoning-based routing decisions). Each LLM call is isolated, role-contextualized, and produces a structured JSON output that the downstream graph nodes can deterministically parse and act upon. The LangChain abstraction layer wraps the OpenAI API, enabling model-agnostic switching to Gemini if needed.

Dependency Diagram

(a) How Prompts Are Constructed and Sent to LLM

```
Agent Module (intake|intelligence|policy|validation|approval)
     |
     | assembles:
     |   SYSTEM constant (static role instructions, hard-coded in module)
     |   user_msg f-string (state-derived: claim, intel, policy, catalog block, …)
     |
     ▼
chat_structured(messages, schema, cfg=settings.cfg_<agent>)
     |
     | injects:
     |   _compact_schema(schema.model_json_schema())  ← strips title/description/default/examples
     |   "You MUST respond with a single JSON object that validates against this JSON schema:\n…"
     |     → merged into the system message (or prepended if absent)
     |
     ▼
chat(merged, temperature=cfg.temperature, max_tokens=cfg.max_tokens, response_format_json=auto)
     |
     | builds:
     |   {model: settings.ilmu_model, messages, temperature, max_tokens,
     |    response_format: {type:"json_object"}  ← only when json mode enabled}
     |
     ▼
OpenAI(base_url=settings.ilmu_base_url, api_key=settings.ilmu_api_key)
     |
     | POST /v1/chat/completions
     ▼
ILMU GLM-5.1 endpoint
```

(b) What Goes Into the Context Window (per agent)

```
Intake context:        SYSTEM (rules) + employee_name/id/team + free_text + receipt_text + attachments
Intelligence context:  SYSTEM + employee_team + extracted claim summary
                       + SubscriptionCatalog.as_prompt_block()  ← all active licences w/ aliases &
seats
                       + fuzzy candidate shortlist (rapidfuzz prefilter, top-N ids+products)
Policy context:        SYSTEM + PolicyStore.as_prompt_block() (rule_id, title, description, severity)
                       + claim summary + intelligence summary
Validation context:    SYSTEM + claim summary + intel summary + policy compliant+violations
Approval context:      SYSTEM + full pydantic JSON dumps of claim, intel, policy, validation
Recorder:              No LLM context (deterministic Python).
request_info:          No LLM context (synthesises from validation.clarifications).
Schema-injection block (always last in system message): compact JSON schema of the expected pydantic
class.
```

(c) How Responses Are Parsed and Passed to the Next Component

```
ILMU response (str)
     |
     ▼
_extract_json(raw):
     1. strip ```json ... ``` fences (FENCE regex)
     2. json.loads(cleaned)         ← happy path
     3. else: regex find first {...} block, json.loads it
     |
     ▼
schema.model_validate(obj)  ← pydantic v2 strict validation
     |           |
     | success   | ValidationError or json.JSONDecodeError
     |           ▼
     |      retry once: append assistant raw + user "your last response failed validation: {err}"
     |      → re-call chat()
     |
     ▼
Typed pydantic instance (IntakeClaim | IntelligenceReport | PolicyReport | ValidationReport |
ApprovalOutcome)
     |
     ▼
written into WorkflowState[<agent>] by the agent node (e.g. {**state, "intake": claim})
     |
     ▼
next node reads via state.get("intake") / state["intake"] (typed access)
     |
     ▼
LangGraph conditional edge functions (_supervisor_route, _post_validation_route) inspect state
and return the next node name
```

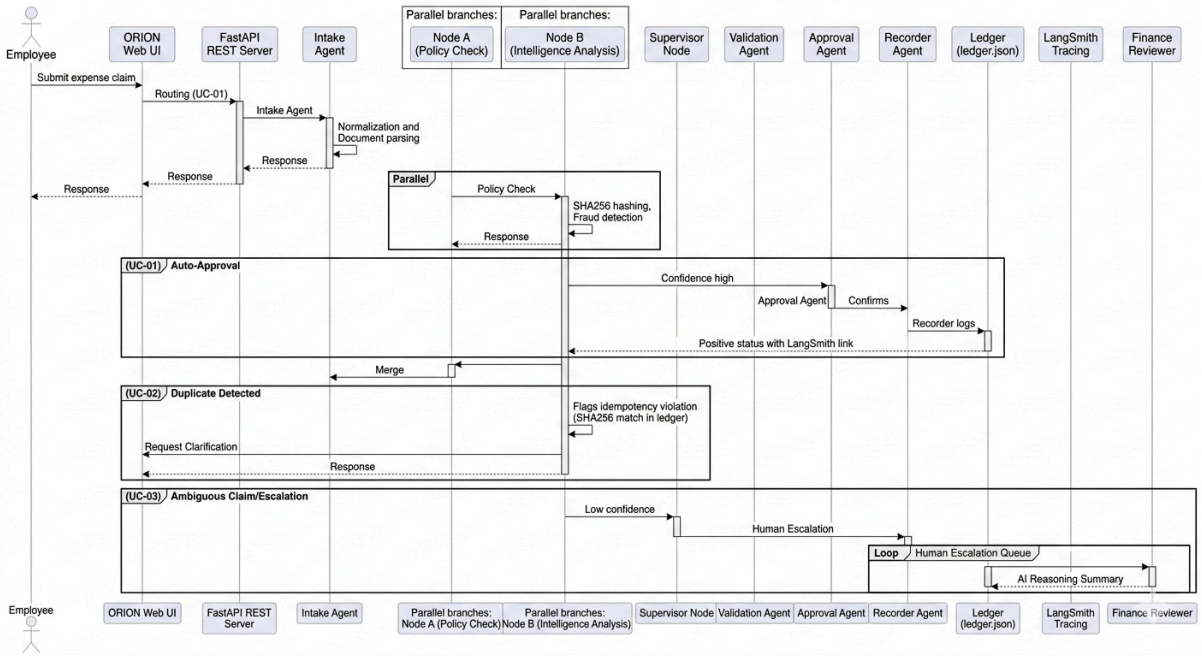
(d) Where Token Limits Are Enforced & Input Is Chunked

```
Per-agent caps (config.py):
  cfg_intake       max_tokens=1400  temp=0.1
  cfg_intelligence max_tokens=1600  temp=0.2
  cfg_policy       max_tokens=1600  temp=0.0
  cfg_validation   max_tokens=900   temp=0.1
  cfg_approval     max_tokens=800   temp=0.0
Schema compaction (llm.py:_compact_schema): drops title/description/default/examples → ~60% schema-
token reduction.
Document parser (tools/document_parser.py):
  HTTP layer enforces settings.max_upload_bytes (8 MB) → 413 if exceeded.
  Parser raises DocumentTooLargeError before extraction → never reaches LLM.
Subscription catalog block:
  Constructed on demand per call — bounded by org_subscriptions.json size.
  No paging in MVP (catalog ≤ ~50 entries assumed).
Policy rulebook:
  Same: full rulebook injected — bounded by policies.json size (~7 rules).
NOT YET IMPLEMENTED (future): chunking long receipts; sliding-window over very large catalogs.
Server-side: ILMU truncates if request exceeds context window — surfaced as APIStatusError.
```

(e) All Major API Calls Between System Components

```
Browser ──────▶ FastAPI:  GET  /                  → static HTML
Browser ──────▶ FastAPI:  GET  /api/health        → {ok, model, langsmith, project, dashboard}
Browser ──────▶ FastAPI:  POST /api/submit        → {claim_id, trace, intake, intelligence, policy,
validation, approval, record, langsmith}
Browser ──────▶ FastAPI:  POST /api/parse-document (multipart) → {filename, text, ...}
Browser ──────▶ FastAPI:  GET  /api/ledger        → {records: [...]}
FastAPI ──────▶ LangGraph:  workflow.invoke(initial_state)  (in-process Python call)
LangGraph ────▶ Agent node functions (intake_node, …)       (in-process)
Agent node ───▶ llm.chat_structured / llm.chat              (in-process)
llm.chat ─────▶ OpenAI SDK ────▶ ILMU /v1/chat/completions (HTTPS, OpenAI-compatible)
@traceable on:
  - llm.chat                        (run_type=llm, name=ilmu.chat)
  - llm.chat_structured             (run_type=llm, name=ilmu.chat_structured)
  - every agent node                (run_type=chain, name=agent.<n>)
  - _run_workflow                   (run_type=chain, name=orion.workflow)
all emit spans ────▶ LangSmith ingest API (langsmith SDK, async batch)
FastAPI ──────▶ LangSmith client.read_run(run_id) → URL of the trace (sync HTTPS)
                                                  → embedded in submit response
Recorder ─────▶ local filesystem: data/ledger.json (append + rewrite)
Intelligence ▶ local filesystem read: data/org_subscriptions.json (cached at module load)
Policy ───────▶ local filesystem read: data/policies.json (cached at module load)
```

Sequence Diagram



Technology Stack

Layer	Technology	Justification
Language	Python 3.12	Pydantic v2 + LangGraph maturity
Web framework	FastAPI 0.115+	Async, pydantic-native, OpenAPI free
Server	uvicorn (standard)	Production-grade ASGI
Workflow engine	LangGraph 1.0.x	Stateful DAG with conditional edges

LLM	ILMU GLM-5.1 (YTL Labs), OpenAI-compatible (GPT 5.4 mini)	Project-mandated model
LLM transport	openai SDK ≥ 1.40	OpenAI-compat shim, swappable
Data validation	pydantic v2 + pydantic-settings	Schema-as-code; .env loading
Observability	LangSmith (langsmith SDK)	Trace tree per request, deep-linkable
Document parsing	pypdf, python-docx	Text extraction without OCR
Fuzzy matching	rapidfuzz	Sub-ms catalog prefilter
UI framework	React 19.0	Component model, hooks, context
Build tool	Vite 6.2	Bundling, HMR, env-var injection at build time
Client-side routing	React Router DOM 7.x	SPA routing for /, /employee, /manager, /finance
Styling	Tailwind CSS 4.x	Utility-first CSS; configured via @tailwindcss/vite plugin
Animation	Motion (Framer Motion) 12.x	Declarative enter/exit animations, drag gestures, scroll transforms
Language Frontend	TypeScript 5.8	Static typing across all frontend source
State management	React Context API	AuthContext (role, username) and ToastContext (notifications)
API communication	Fetch API — typed	api/client.ts wraps all five backend endpoints with TypeScript request/response types
Persistence	Append-only JSON file (MVP)	Zero infra; SQLite/Postgres in Phase 2
Tests	pytest 8 + pytest-cov	Coverage gate at 80%
CI	GitHub Actions (ubuntu-latest, py 3.12)	PR gate + nightly cron
Secrets	GitHub Secrets + .env (local)	Per-environment isolation

Key Data Flows & Database Schema

Data Flow Workflow

```
Employee ——— HTTP POST ———►┌──────────────┐
(free_text,                │ /api/submit  │
 receipt_text,             │ (FastAPI)    │
 attachments)              └──────┬───────┘
                                  │
                                  ▼
                   ┌──────────────────────┐
                   │ WorkflowState (TypedDict)│◄──────────┐
                   └──────┬───────────────┘            │
                          │                            │
                          ▼                            │
                     [intake_node]                     │
                (LLM: free_text+receipt → IntakeClaim) │
                          │                            │
                          ▼                            │
                [intelligence_node] ◄——— data/org_subscriptions.json
                (LLM: claim+catalog → IntelligenceReport)
                          │                            │
                          ▼                            │
                     [policy_node] ◄———— data/policies.json
                 (LLM: claim+intel → PolicyReport)    │
                          │                            │
                          ▼                            │
                  ┌——— _supervisor_route ———┐          │
          hard_block&conf≥0.7?              otherwise  │
                  │                          │         │
                  ▼                          ▼         │
            [approval_node]            [validation_node]
                  │                          │         │
                  │                ready_for_decision? │
                  │                ┌——yes——no——┐       │
                  │                ▼          ▼       │
                  │          [approval_node]  [request_info_node]
                  │                │          │       │
                  └————►[recorder_node] ◄————┘        │
                                │                     │
                                ▼                     │
                        data/ledger.json (append) ———┘
                                │
                                ▼
                 HTTP 200 — { trace, agent reports,
                              approval, langsmith.run_url }
                                │
                                ▼
                        Employee UI — visual workflow,
                        decision pill, "sent back" panel
```

DFD Compliance Notes

- *No Black Hole: Every process that receives data produces an output*
- *No Miracle: No data store is written to without a corresponding process write.*
- *Data consistency: WorkflowState is the single source of truth as a claim traverses the graph, no process reads directly from ledger.json mid-workflow.*

Normalised Database Schema (ERD) , 3NF

```
EMPLOYEE (1)──────────(M) CLAIM (1)──────────(M) AGENT_RUN (1)──────(M) AGENT_OUTPUT
                              |
                              | (1)
                              |
                              ▼ (M)
                        POLICY_VIOLATION ──────────► POLICY (M:1)
                              |
                              |
                        DUPLICATE_MATCH ──────────► ORG_SUBSCRIPTION (M:1)
EMPLOYEE
  PK employee_id        VARCHAR
     name               VARCHAR
     team               VARCHAR
CLAIM
  PK claim_id           VARCHAR  -- "CLM-XXXXXXXX"
  FK employee_id        VARCHAR  → EMPLOYEE.employee_id
     vendor             VARCHAR
     product            VARCHAR
     category           ENUM
     amount_myr         DECIMAL(12,2)
     currency_original  CHAR(3)
     amount_original    DECIMAL(12,2)
     billing_period     ENUM
     purchase_date      DATE
     business_justification TEXT
     extraction_confidence FLOAT
     final_decision     ENUM     -- ApprovalDecision values
     final_reason       TEXT
     decision_confidence FLOAT
     created_at         TIMESTAMPTZ
     terminal_at        TIMESTAMPTZ
     langsmith_run_id   UUID NULL
AGENT_RUN
  PK run_id             UUID
  FK claim_id           VARCHAR  → CLAIM.claim_id
     agent_name
```

```
ENUM('intake','intelligence','policy','validation','approval','recorder','request_info')
     started_at         TIMESTAMPTZ
     ended_at           TIMESTAMPTZ
     trace_index        INT      -- order within the claim
AGENT_OUTPUT
  PK output_id          UUID
  FK run_id             UUID     → AGENT_RUN.run_id
     output_jsonb       JSONB    -- the schema-validated pydantic dump
POLICY  (mirrors data/policies.json, but a 1st-class table)
  PK rule_id            VARCHAR  -- "POL-001"
     title              VARCHAR
     description        TEXT
     severity           ENUM('warn','block')
     applies_to         VARCHAR[]
     active             BOOL
POLICY_VIOLATION
  PK violation_id       UUID
  FK claim_id           VARCHAR  → CLAIM.claim_id
  FK rule_id            VARCHAR  → POLICY.rule_id
     description        TEXT     -- contextualised at evaluation time
ORG_SUBSCRIPTION
  PK org_sub_id         VARCHAR  -- "ORG-SUB-001"
     vendor             VARCHAR
     product            VARCHAR
     category           ENUM
     seats_total        INT
     seats_used         INT      -- seats_available is derived (NOT stored — 3NF)
     owner_team         VARCHAR
     renewal_date       DATE
ORG_SUBSCRIPTION_ALIAS  (separated to keep 1NF)
  PK (org_sub_id, alias)
  FK org_sub_id         → ORG_SUBSCRIPTION.org_sub_id
     alias              VARCHAR
DUPLICATE_MATCH
  PK match_id           UUID
  FK claim_id           VARCHAR  → CLAIM.claim_id
  FK org_sub_id         VARCHAR  → ORG_SUBSCRIPTION.org_sub_id
     similarity_score   FLOAT
     reasoning          TEXT
NOTIFICATION
  PK notification_id    UUID
  FK claim_id           VARCHAR  → CLAIM.claim_id
     recipient          VARCHAR  -- "employee:E123" / "role:finance_controller"
     channel            ENUM('ledger','email','slack')
     sent_at            TIMESTAMPTZ
```

Functional Requirements & MVP Scope

Minimum Viable Product Features

#	Feature	Description
1	Free-text submission endpoint	Employees describe their reimbursement in plain English (no fixed form fields) via POST /api/submit; the system handles structuring downstream.
2	Document upload + parse (PDF/DOCX)	Accepts PDF / DOCX / TXT up to 8 MB; extracted text is piped into the same context window the Intake Agent reads, removing manual data entry.
3	Intake Agent (structured extraction)	LLM-driven conversion of unstructured claim + receipt into a strictly typed IntakeClaim (vendor, product, category, amount, billing period, justification) with a self-reported confidence score and explicit missing_fields.
4	Intelligence Agent (semantic duplicate detection)	Compares the request against the org's active SaaS catalog using rapidfuzz prefilter + LLM judgment over aliases and seat availability, so name variations ("ChatGPT Plus" vs "OpenAI GPT") still resolve to the same licence.
5	Intelligence Agent (alternative recommendation)	When a duplicate has free seats or a cheaper tier exists, the agent recommends that path instead of new spend, with estimated savings in MYR.
6	Policy Agent (rulebook compliance check)	Evaluates the claim against data/policies.json (POL-001..POL-007) and emits a PolicyReport distinguishing blocking vs warning severities, scoped by category.
7	Adaptive routing supervisor	Conditional edge inspects intelligence + policy + intake confidence and skips the validation step when a hard block is already certain, turning the workflow from a fixed pipeline into a stateful adaptive engine.
8	Validation Agent (ambiguity handling)	Generates 1–3 targeted clarifying questions when the case file lacks signal, instead of forcing a low-confidence reject.
9	request_info terminal branch	Synthesises a REQUEST_INFO outcome (deterministic, no extra LLM call) packaging the validation questions for the employee.
10	Approval Agent (final 5-way decision)	Synthesises the full case file and emits one of auto_approve / auto_reject / escalate_manager / escalate_finance / request_info with reason, next action, approver role, and self-confidence.
11	Recorder Agent (append-only audit ledger)	Every terminal decision is persisted to data/ledger.json with claim id, employee id, amount, decision, timestamp, and notification list, fulfilling the auditability requirement.
12	LangSmith trace deep-link in response	Every /api/submit response carries langsmith.run_url, letting an admin pivot from a single decision into the full agent reasoning tree.
13	Health endpoint	GET /api/health reports model id, LangSmith live status, and dashboard URL used by smoke tests, CI, and SRE checks.
14	Ledger read endpoint	GET /api/ledger returns recent decisions for the demo UI's history view and for offline auditors.
15	Role-Based Login Page	Landing page where users select their role (Employee, Manager, or Finance Controller) and enter credentials before being routed to their dedicated dashboard.
16	Employee Dashboard & Claim Submission	Bento-grid dashboard with a 3-step modal wizard for submitting reimbursement claims in natural language, attaching a PDF or Word receipt (auto-parsed), and viewing the AI decision in real time.

17	Manager & Finance Dashboards	Manager dashboard with a swipe-to-approve card interface for escalated claims; Finance dashboard with a searchable ledger table, policy reference panel, and spend analytics.
18	Typed API Client & Real-Time Feedback	Centralized api/client.ts connecting all five backend endpoints with TypeScript types, 20-second ledger polling, and a toast notification system surfacing submission results and errors across all dashboards.
19	Offline-deterministic test path	A stub LLM in tests/conftest.py mirrors scripts/smoke.py canned outputs so the full graph is testable without ILMU credentials, enabling free PR CI.
20	Live regression nightly	pytest --runlive runs the full graph against real ILMU + LangSmith on a cron schedule with decision-band assertions and a 120s SLA per scenario.

Non-Functional Requirements

Quality	Requirement	Implementation
Performance	/api/submit end-to-end latency	p50 < 30s, p95 < 120s
	/api/health, /api/ledger	p95 < 200ms
Reliability	Decision returned for every accepted submission	99%, remaining 1% surfaces error
	Ledger durability: no lost decisions on process restart	100% (fsync on append)
Scalability	Concurrent submissions (single-process MVP)	5 concurrent / process
Observability	Every agent step traced	100% when LangSmith on
Security	API keys never logged or echoed	enforced by review
	Employee free-text never executed as instruction	architectural
Compliance	Every decision auditable in ledger with timestamp + reason	100%
Maintainability	Line coverage on app/	≥ 80% (CI-enforced)
Portability	Runs on Python 3.12, Linux + Windows	verified in CI
Cost	Token budget per claim	≤ 10k input + 6k output (sum of max_tokens)
Usability	Demo UI single page, no build step	static HTML

Out of Scope / Future Enhancements

- Direct ERP or payroll system integration (e.g., SAP, Workday)
- Mobile application interface for employees.
- Real-time push notifications (email or SMS) to employees on claim status changes.
- Multi-currency conversion and foreign exchange rate handling.
- Role-based access control (RBAC) UI for managing reviewer permissions.
- OCR for handwritten receipts or non-digital image formats.
- Multi-tenant / multi-organization support in v1 architecture.

- Machine learning model fine-tuning pipeline based on historical decision data.

Monitor, Evaluation, Assumptions & Dependencies

Technical Evaluation

Grayscale Rollout & A/B Testing Strategy

- **Stage 0:** Shadow mode: deploy alongside the current manual reimbursement flow. Submissions still go to humans; Orion runs in parallel and writes to ledger only. Compare Orion's decision against the human's after 2 weeks.
- **Stage 1:** 10% live cohort: route 10% of submissions (hash on employee_id) through Orion as the primary. Manual override is always available.
- **Stage 2:** 50%: expand on green metrics: ≥ 90% agreement with human-baseline (rolling-5 nights), zero P0 incidents, manual-override rate < 15%.
- **Stage 3:** 100%: full cutover; manual queue retained for escalate_* and request_info decisions.
- **A/B:** rollouts can use config flag ilmu_model (e.g., glm-5.1 vs glm-5.1-tuned) per cohort. Cohort assignment passed in submission metadata; LangSmith tags trace with cohort id; A/B compared on (a) decision-band agreement, (b) p95 latency, (c) override rate.

Emergency Rollback (ER) & Golden Release Strategy

- **Golden release tag:** every successful nightly+manual-QA SHA is tagged golden-YYYY-MM-DD after passing the deployment threshold.
- **Rollback trigger (any one):**
  - P0 condition met.
  - Override rate > 30% over any 1-hour window.
  - LangSmith error rate > 5% over 5 min.
- **Rollback action:** redeploy last golden-* tag via single workflow dispatch; this is a containerised image redeploy (no DB migration in MVP since ledger is append-only JSON). Total recovery time target: < 10 min.
- **Forward-fix-only:** code changes always go forward through the main >> tag. Hotfixes branch from latest **golden-***, fast-track CI, tag **hotfix-***, merge back into main.

Priority Matrix

Priority	Trigger Condition	Detection	Action
P0 Halt	/api/submit returns 5xx > 5% over 5 min, or any decision is auto_approve for amount > MYR 5000	LangSmith error rate alarm + ledger audit query	Page on-call; toggle feature flag to maintenance page; rollback to previous tag

P1 Critical	Live test pass rate < 75% on a single nightly run, OR coverage drops below 75%	nightly workflow failure	Block next deploy; root-cause within 24h
P2 High	Schema-validation retries > 10% of LLM calls, OR live SLA p95 > 90s	LangSmith run-tree analysis	Tune prompts / temperature; re-run nightly
P3 Medium	Single decision-band miss in expected-payload test	nightly diff against expected.yaml	Investigate prompt drift; widen band only if intentional
P4 Low	Deprecation warning, doc rot, low-coverage module identified	weekly review	Backlog ticket

Monitoring

Orion's monitoring strategy is anchored on LangSmith as the single pane of glass for every workflow execution. Each reimbursement submission is wrapped in a root trace, and every agent step plus every underlying language-model call emits its own child span. The result is a fully linked run-tree that surfaces per-agent latency, token usage, prompt and response payloads, JSON-validation retries, and the exact path the supervisor took through the conditional graph, so when a claim is rejected or escalated, an operator can click a single deep-link from the UI and replay the decision end-to-end. At the application boundary, a lightweight health endpoint exposes a liveness probe reporting model identity, LangSmith connectivity, and project name, which is the contract a future uptime monitor (such as Pingdom or UptimeRobot) would poll on a fixed cadence to confirm the service is reachable.

CI/CD observability runs on two complementary surfaces. The pull-request gate runs on every push and reports lint cleanliness, code coverage against an enforced minimum, and unit/integration pass-rate before any change can merge. The nightly workflow runs on a scheduled cadence (and on demand) to exercise the live language-model and tracing path against a fixed scenario suite, which catches model-drift and upstream-API regressions that mocked tests cannot see. Together they form a two-tier early-warning system: the PR gate protects the codebase from regressions introduced by developers, while the nightly run protects the production behaviour from regressions introduced by the model provider.

Failure-class telemetry is collected at three layers. Transient API failures, timeouts, server errors, and rate-limit responses, are counted and retried with exponential backoff inside the language-model client, so intermittent upstream weather does not show up as a user-visible outage. Structured-output validation failures, where the model returns malformed JSON or violates a schema, are caught and retried once with the validator error fed back to the model as a correction prompt, and any remaining failure is surfaced as a typed error rather than a silent fall-through. Workflow-level exceptions are

caught at the API boundary and returned as a clean HTTP error with a stack-trace written to standard output, which on a hosted deployment feeds directly into the platform's log aggregator and could later be wired into an error-tracking service such as Sentry.

The current gap, tracked on the project board as a follow-up, is the absence of a dedicated metrics pipeline (for example Prometheus and Grafana) and of synthetic SLA probes hitting the deployed submission endpoint from outside the cluster. Until those are in place, the deployment-gate row for live API performance remains only partially measured: offline-proxy latency is green, but the production service-level objective is pending the first scheduled run of the nightly suite against the deployed environment.

Assumptions

The following operational and environmental conditions are assumed to be valid at the time of development and demonstration:

- Employees submit in English; receipts are in MYR or a major currency convertible by the LLM.
- ILMU GLM-5.1 endpoint exposes an OpenAI-compatible Chat Completions interface (/v1/chat/completions) with optional JSON mode.
- LangSmith project exists and the key has write access to it.
- The org subscriptions list (data/org_subscriptions.json) is refreshed out-of-band, Orion does not write back to it.
- Ledger volume is low enough (< 10k records) that a single JSON file is acceptable for MVP.
- One concurrent FastAPI worker; no horizontal scaling assumed.
- Free-text and receipt uploads are ≤ 8 MB (max_upload_bytes).
- All policies fit comfortably in a single LLM context; today the rulebook is ~7 rules.

External Dependencies

Tool	Purpose	Failure Mode	Fallback
ILMU GLM-5.1 API	Central LLM reasoning engine all 6 inference calls per workflow depend on this	Outage / rate-limit / 5xx	Built-in retry+backoff; surfaced as 500 to client
LangSmith	Trace observability	Outage	Tracing silently disabled (langsmith_is_live() returns False); workflow continues
LangGraph	Workflow DAG execution	Pinned minor	Locked in requirements.txt

pydantic	Schema validation	Breaking minor	Locked in requirements.txt
OpenAI SDK	HTTP transport to ILMU	Breaking change	Locked; wrapper isolates surface
FastAPI / uvicorn	HTTP server	Standard web server failure	Container restart
pypdf, python-docx	Document text extraction	Parsing exception	400 returned to client
rapidfuzz	Catalog fuzzy prefilter	Performance only	Empty candidate list >> LLM still has full catalog

Project Management & Team Contributions

Project Timeline

Phase	Window	Duration
P1 — Ideation & PRD	Thu 16 Apr 8 PM → Sat 18 Apr 6 PM	~2 days
P2 — System Analysis & Design	Sat 18 Apr 6 PM → Mon 20 Apr 11 PM	~2.5 days
P3 — Implementation (code)	Mon 20 Apr → Thu 23 Apr 8 PM	~3.5 days
P4 — QA Testing & Documentation	Wed 22 Apr (parallel) → Fri 24 Apr 6 PM	~2.5 days
P5 — Pitch Deck	Thu 23 Apr 8 PM → Sat 25 Apr 2 PM	~2 days
P6 — Demo / Pitch Video (10 min)	Sat 25 Apr 2 PM → Sun 26 Apr 11:59 PM	~1.5 days

Team Members Role

Name	Role	Details
David Kurniawan	Tech Lead / Backend & LangGraph Architect	Owns the agent topology, LLM service layer (app/llm.py), schema contracts, and CI/CD. Decides graph routing rules, prompt structure, and the offline/live test split. Primary author of PRs to feats/ci-cd and feat/fix-agents.

Vanessa Serenina Prawirayasa	AI / Prompt Engineer	Owns per-agent system prompts (intake, intelligence, policy, validation, approval), tunes per-agent temperature/max_tokens in app/config.py, curates the expected.yaml decision bands, and analyses LangSmith traces to drive prompt iteration after each nightly run.
Jason Clarence Setyabudi	QA / Test Engineer	Owns the Requirement Traceability Matrix, tests/conftest.py stub LLM, all unit + integration tests, coverage thresholds, the GitHub Actions workflows, and the nightly regression triage. Sign-off on every release tag.
Darren Cornelius Suwandi	Frontend / UX & Demo Lead	Owns app/web/index.html, workflow diagram CSS/JS, decision pill states, sent-back-to-employee callout, document upload UX, LangSmith deep-link rendering. Drives the demo video, slide deck visuals, and stakeholder walkthrough.

Recommendations

- Promote the supervisor to a first-class LLM agent. Today _supervisor_route is a hand-coded heuristic (hard_block AND conf >= 0.7). Phase 2 should let a small, fast LLM call decide the route, but only after we add a hard-iteration cap to prevent loops. This unlocks more nuanced routing (e.g., "ask intel one more question first") that the current binary heuristic cannot express.
- Move the ledger off JSON before it bites. Append-only JSON works at low volume but corrupts under concurrent writes and has no query layer. Migrate to SQLite (single-file, zero-ops) for MVP+, with the 3NF schema in §14 as the target. Keep the JSON file as a debug export.
- Split the LLM service from the OpenAI SDK with a Protocol. Define an LLMService Protocol in app/llm.py, give the ILMU implementation as one concrete class. This makes the test suite swappable without monkey-patching imports across every agent module (the current pattern in tests/conftest.py:stub_llm).
- Add idempotency keys at the API boundary. claim_id is generated server-side, so a network retry produces a duplicate workflow. Accept an optional Idempotency-Key header on /api/submit and short-circuit to the cached response, critical before any non-trivial production rollout.
- Externalise prompts. System prompts are inline string constants today. Move them into versioned files under prompts/ so prompt changes are reviewable, diffable, and roll-back-able independently of code.
- Treat policies as data, not code. Today data/policies.json is read fresh, but the LLM prompt embeds the rule text verbatim. Add a thin policy-engine layer that emits deterministic compliance checks for the unambiguous rules (amount thresholds, missing receipt, missing

justification) and only invokes the LLM for the genuinely judgment-bound cases. This is faster, cheaper, and far more auditable.

- Introduce a hard iteration / cost guard. No agent should be able to call the LLM more than N times per claim. Encode this as a counter in WorkflowState and as a per-claim token budget enforced in app/llm.py. Cheap insurance against future graph-cycle bugs.
- Add an evaluator harness for prompt regressions. Today nightly tests assert decision bands. Add a separate eval that runs the same scenarios on each prompt change, scores them with a reference LLM, and gates the PR, catches drift the band-tests miss when the LLM lands in-band but the reasoning degrades.
- Separate the "reviewer override" data path now, not later. Even before building the reviewer UI, add an override column / field so when humans start reversing decisions in Phase 2, the data shape is already there to mine for prompt improvement (active learning).
- Document the trust boundary explicitly. Employee free-text and receipt text are untrusted input; org subscription catalog and policy rules are trusted configuration. The agent prompts should make this distinction visible in their system messages, and the architecture document should call it out so future maintainers don't accidentally let user input become instruction context.
- Pin a golden ILMU model snapshot. settings.ilmu_model = "ilmu-glm-5.1" floats. For releases, pin to a dated snapshot id and update deliberately, this turns a silent quality regression into a deliberate, reviewable change.
- Build the audit-export endpoint before you need it. A GET /api/audit/export?since=... returning CSV or JSON of ledger entries is one afternoon of work and removes the future scramble when finance / compliance asks for a dump.